

# Memento

Památka



# Hlavní idea

**Memento** (Památka) je behaviorální návrhový vzor (vzor související s chováním), který nám umožňuje uložit a obnovit předchozí stav objektu, aniž byste odhalili podrobnosti jeho implementace.

Návrhový vzor memento ukazuje, jak řešit problém externalizace, uchování a obnovení vnitřního stavu objektu bez porušení **principu zapouzdření**.





# Problém

---

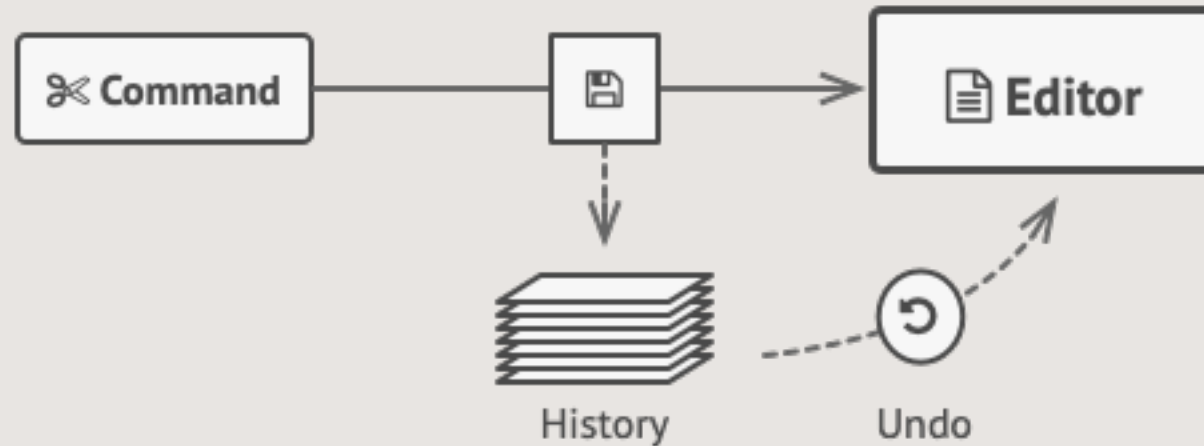
Představte si, že vytváříte aplikaci textového editoru. Kromě jednoduchých úprav textu může váš editor formátovat text, vkládat vložené obrázky atd.

V určitém okamžiku jste se rozhodli umožnit uživatelům vrátit zpět všechny operace provedené s textem. Tato funkce se v průběhu let stala tak běžnou, že v dnešní době lidé očekávají, že ji bude mít každá aplikace. Pro implementaci jste zvolili přímý přístup. Před provedením jakékoli operace zaznamená aplikace stav všech objektů a uloží jej do nějakého úložiště. Později, když se uživatel rozhodne vrátit akci, aplikace načte nejnovější snímek (snapshot) z historie a použije jej k obnovení stavu všech objektů.



# Problém

---



*Před provedením operace aplikace uloží snímek stavu objektů, který lze později použít k obnovení objektů do předchozího stavu.*



# Problém

---

Zamysleme se nad těmi stavovými snímky (snapshots). Jak přesně byste každý z nich vyrobili? Pravděpodobně budete muset projít všechna pole v objektu a zkopírovat jejich hodnoty do úložiště. To by však fungovalo pouze v případě, že by objekt měl docela uvolněná omezení přístupu ke svému obsahu. Bohužel většina skutečných objektů nedovolí ostatním do nich nahlédnout tak snadno a skryje všechna významná data v soukromých polích.



# Problém

---

Tento problém prozatím ignorujte a předpokládejme, že se naše objekty chovají jako hippies: preferují otevřené vztahy a udržují svůj stav veřejný. I když by tento přístup vyřešil okamžitý problém a umožnil vám vytvářet snímky stavů objektů podle libosti, stále však zde existuje několik vážných problémů. V budoucnu se můžete rozhodnout refaktorovat některé třídy editorů nebo přidat nebo odebrat některá pole. Zní to jednoduše, ale vyžadovalo by to také změnu tříd odpovědných za kopírování stavu dotčených objektů.

**private** = can't copy

**public** = unsafe

| Editor        |
|---------------|
| - text        |
| - cursorPos   |
| - selection   |
| - currentFont |
| - styles      |
| ...           |
| ...           |



# Problém

---

Ale je toho víc. Podívejme se na skutečné „snímky“ stavu našeho editoru. Jaké údaje obsahuje? Minimálně musí obsahovat skutečný text, souřadnice kurzoru, aktuální pozici posouvání atd. Chcete-li vytvořit snímek, musíte tyto hodnoty shromáždit a vložit do nějakého kontejneru.

S největší pravděpodobností budete ukládat spoustu těchto kontejnerových objektů do nějakého seznamu, který by představoval historii. Proto by kontejnery pravděpodobně skončily jako objekty jedné třídy. Třída by neměla téměř žádné metody, ale spoustu polí, která odrážejí stav editoru.



# Problém

---

Chcete-li umožnit jiným objektům zapisovat a číst data do/z snímku, budete pravděpodobně muset zveřejnit jeho pole. To by odhalilo všechny stavy editoru, soukromé nebo ne. Ostatní třídy by se staly závislými na každé malé změně třídy snapshot, ke které by jinak došlo v rámci soukromých polí a metod, aniž by to ovlivnilo vnější třídy.

Vypadá to, že jsme se dostali do slepé uličky: buď odkryjete všechny vnitřní detaily tříd, nebo omezíte přístup k jejich stavu, čímž znemožníte vytváření snímků. Existuje nějaký jiný způsob, jak implementovat "zpět"?



# Řešení

---

Všechny problémy, které jsme právě zažili, jsou způsobeny zapouzdřením. Některé objekty se snaží dělat víc, než by měly. Aby shromáždili data potřebná k provedení nějaké akce, proniknou do privátního prostoru jiných objektů, místo aby tyto objekty nechali provést skutečnou akci.

Vzor Memento deleguje vytváření snímků stavu na skutečného vlastníka tohoto stavu, *původní* objekt. Místo toho, aby se jiné objekty pokoušely zkopírovat stav editoru z „venku“, může snímek vytvořit samotná třída editoru, protože má plný přístup ke svému vlastnímu stavu.



## Řešení

---

Vzor navrhuje uložit kopii stavu objektu do speciálního objektu zvaného **Memento** . Obsah **Memento** není přístupný žádnému jinému objektu kromě toho, který jej vytvořil. Ostatní objekty musí komunikovat s upomínkami pomocí omezeného rozhraní, které může umožnit načíst metadata snímku (čas vytvoření, název provedené operace atd.), nikoli však původní stav objektu obsažený ve snímku.



## Řešení

---

Taková omezující politika vám umožňuje ukládat upomínky do jiných objektů, obvykle nazývaných **správci**. Vzhledem k tomu, že správce pracuje s mementem pouze prostřednictvím omezeného rozhraní, není schopen manipulovat se stavem uloženým v mementu. Současně má původce přístup ke všem polím uvnitř mementa, což mu umožňuje libovolně obnovit jeho předchozí stav.



# Řešení

---

V našem příkladu textového editoru můžeme vytvořit samostatnou třídu historie, která bude fungovat jako správce. Zásobník upomínek uložených uvnitř správce se zvětší pokaždé, když se editor chystá provést operaci. Tento zásobník můžete dokonce vykreslit v uživatelském rozhraní aplikace a zobrazit tak uživateli historii dříve provedených operací.

Když uživatel spustí vrácení zpět, historie získá nejnovější upomínku ze zásobníku a předá je zpět editoru s žádostí o vrácení zpět. Vzhledem k tomu, že editor má k upomínce plný přístup, mění svůj vlastní stav s hodnotami převzatými z mementa.



# Výhody a nevýhody

---

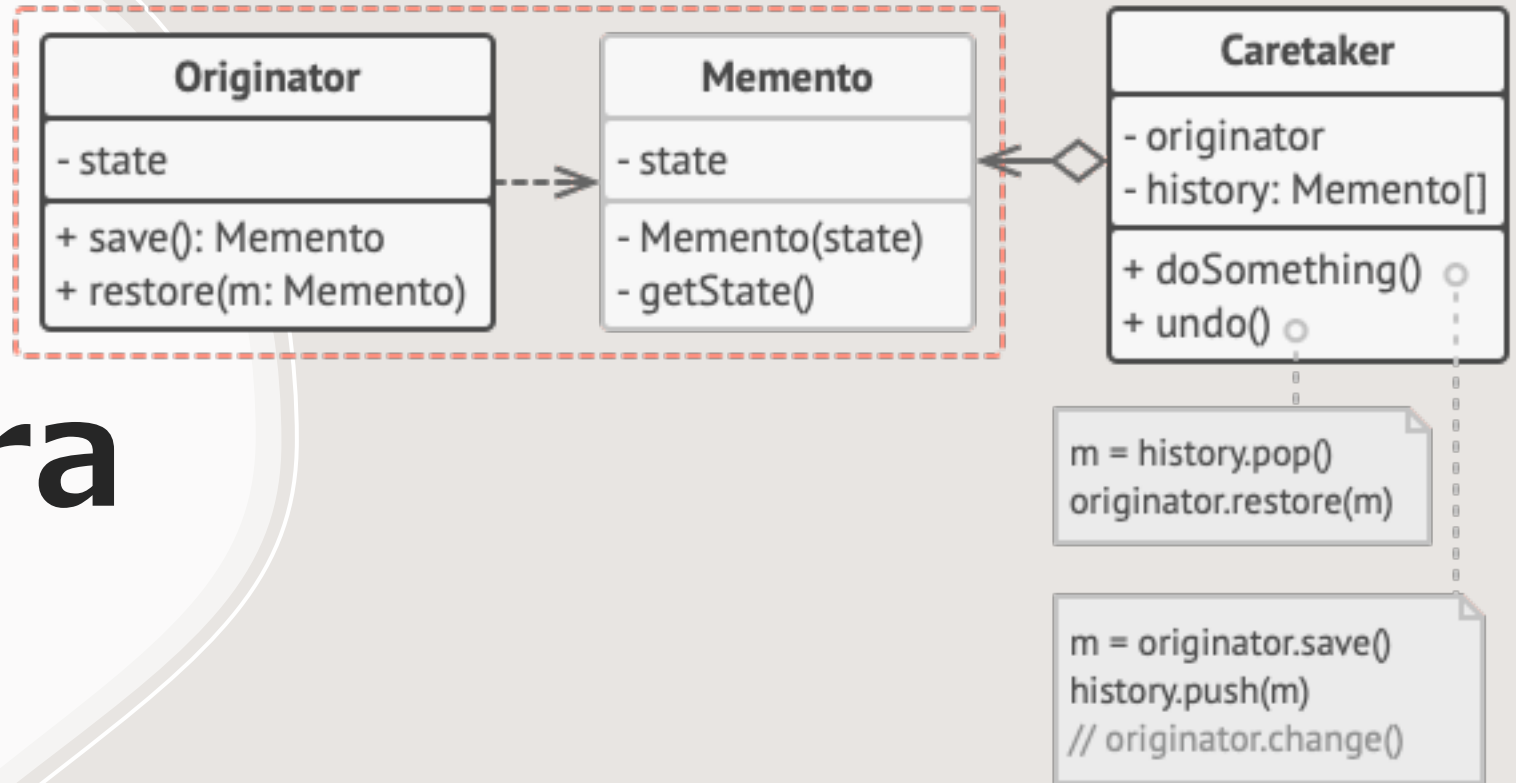


- Můžete vytvářet snímky stavu objektu, aniž byste narušili jeho zapouzdření.
- Kód původce můžete zjednodušit tím, že ponecháte správci udržovat historii stavu původce.

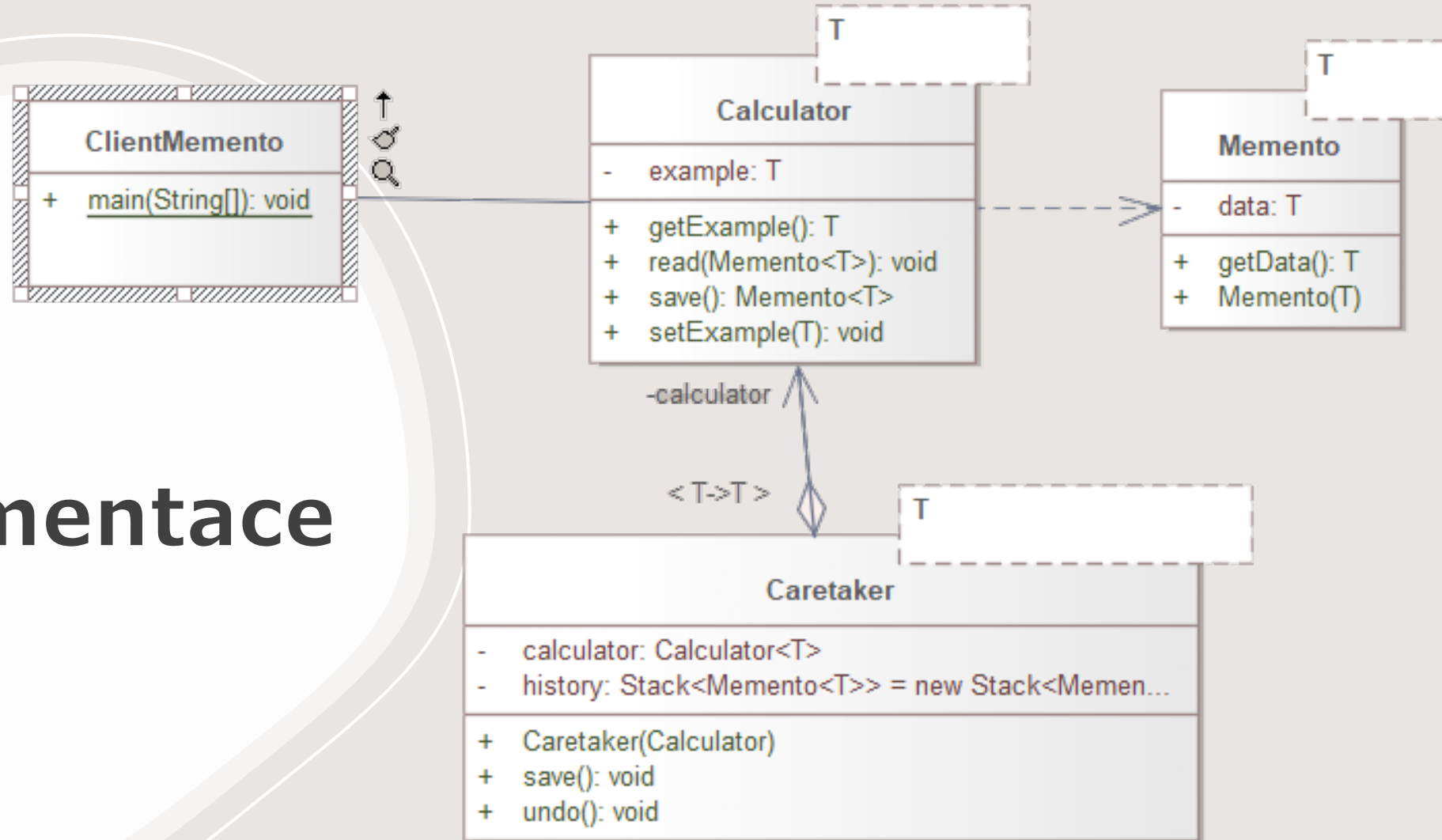


- Pokud klienti vytvářejí upomínky příliš často, může aplikace spotřebovat spoustu paměti RAM.
- Správci by měli sledovat životní cyklus původce, aby byli schopni zničit zastaralá upomínková díla.

# Struktura



# Implementace





```
public class Memento<T>
{
    private T data;

    public Memento(T data) {
        this.data = data;
    }

    public T getData() {
        return data;
    }
}
```

```
//Originator
public class Calculator<T> {

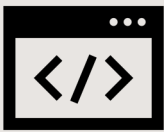
    private T example;

    public Memento<T> save() {
        return new Memento(example);
    }

    public void read(Memento<T> memento) {
        example = memento.getData();
    }

    public void setExample(T example) {
        this.example = example;
    }

    public T getExample() {
        return example;
    }
}
```



```
import java.util.Stack;

public class Caretaker<T> {
    private Calculator<T> calculator;
    private Stack<Memento<T>> history = new Stack<Memento<T>>();

    public Caretaker(Calculator calculator) {
        this.calculator = calculator;
    }

    public void save() {
        history.push(calculator.save());
    }

    public void undo() {
        calculator.read(history.pop());
    }
}
```

```
public class ClientMemento {

    public static void main(String[] args) {
        Calculator calculator = new Calculator();
        Caretaker<String> historie = new Caretaker<>(calculator);
        calculator.setExample("1 + 1");
        System.out.println(calculator.getExample());
        historie.save();

        calculator.setExample("2 + 2");
        System.out.println(calculator.getExample());
        historie.save();

        calculator.setExample("3 + 3");
        System.out.println(calculator.getExample());
        historie.save();

        historie.undo();
        System.out.println("undo:" + calculator.getExample());

        historie.undo();
        System.out.println("undo:" + calculator.getExample());
    }
}
```



## Zdroje

---

<https://refactoring.guru/design-patterns>

<http://voho.eu/wiki/navrhovy-vzor/>

<https://www.oodesign.com/>